



KG COLLEGE OF ARTS AND SCIENCE

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore - 641 035

LAB MANUAL

R Programming Lab

B.Sc. IT

2024 BATCH

Core Lab V: R Programming

Course Code	Course Name	Category	Hours/Week	Credits
24BIT52P	Lab: R Programming	Core Lab-V	5	3

S.No	List of Programs
.	
	Sample Programs
1	Write an R program to demonstrate expressions, variables, and basic functions .
2	Create vectors, matrices, lists, and data frames in R and perform basic operations.
3	Create matrices and perform addition, multiplication, transpose and determinant
4	Create factor variables and analyze categorical data.
5	Read a CSV file and display structure and summary.
6	Read and write data from Excel, JSON, and text files.
7	Perform data preprocessing on a dataset: handle missing values and omit null values.
8	Create a data frame and explore using str(), summary(), head()
9	Compute mean, median, mode, variance, and standard deviation.
10	Validate regression assumptions (linearity, normality, homoscedasticity)
11	Detect outliers using boxplots and statistical methods.
12	Build a binary classification model using glm() with binomial family.
13	Plot and interpret decision tree structure.
14	a)Apply k-means clustering and visualize clusters.

	b)Compare clustering results using different values of k. c)Perform hierarchical clustering and plot dendrogram.
15	a)Generate association rules using arules package. b)Identify frequent itemsets using Apriori algorithm. c)Plot association rules using graphs.
	Total Hours 75
Text Books	
1.	Seema Acharya (2024),”Data Analytics using R” (8th Edition), McGraw Hill Education.
2.	Daniel Bell (2019),”R Programming: A Step-by-Step Guide for Absolute Beginners” (1 st Edition), GUZZLER MEDIA.
Reference Books	
1.	Peng, R. D (2014). R programming for Data Science. Lean Publishing.
2.	Prajapati, V. (2013). Big Data Analytics with R and Hadoop. Packt Publishing.
Web Resources (Swayam/NPTEL)	
1.	Exploratory Data Analysis for Data Science with R Software — NPTEL (IIT Kanpur)
2.	Business Analytics and Data Mining Modeling Using R
3.	Descriptive Statistics with R Software

Experiment 1:

Demonstration of Expressions, Variables and Basic Functions in R

Aim

To write an R program that demonstrates the use of expressions, variables and basic built-in functions.

Algorithm

Step 1 : Start the process

Step 2 : Open R Studio and select File → New File → RScript

Step 3: Declare two variables and assign values.

Step 4: Use arithmetic expressions and functions

Step 5: Print the results.

Step 6: Stop the process.

Program

```
# Demonstration of Expressions, Variables and Basic Functions
```

```
# Variables
```

```
a <- 20
```

```
b <- 10
```

```
# Arithmetic Expressions
```

```
sum_result <- a + b
```

```
difference <- a - b
```

```
product <- a * b
```

```
quotient <- a / b
```

```
power <- a^2
```

```
# Display Results
```

```
print(paste("Addition =", sum_result))
```

```
print(paste("Subtraction =", difference))
```

```
print(paste("Multiplication =", product))
print(paste("Division =", quotient))
print(paste("Square of a =", power))

# Basic Functions
numbers <- c(5, 10, 15, 20, 25)

print(paste("Square Root of 25 =", sqrt(25)))
print(paste("Mean =", mean(numbers)))
print(paste("Maximum =", max(numbers)))
print(paste("Minimum =", min(numbers)))
print(paste("Length =", length(numbers)))
print(paste("Sum =", sum(numbers)))
```

Output

```
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2
Square of a = 400
Square Root of 25 = 5
Mean = 15
Maximum = 25
Minimum = 5
Length = 5
Sum = 75
```

Result

Thus, the R program demonstrating expressions, variables, and basic functions was executed successfully and the results were verified.

Experiment 2

Creating Vectors, Matrices, Lists and Data Frames in R

Aim

To create vectors, matrices, lists and data frames in R and perform basic operations on them.

Algorithm

Step 1 : Start the process

Step 2 : Open R Studio and select File→New File→ RScript

Step 3: Declare vectors using c() and assign values.

Step 4: Declare Matrices with matrix function and assign values.

Step 5: Declare List with list function and assign values.

Step 6: Declare Data Frame with data.frame function and assign values.

Step 7: Print the results.

Step 8: Stop the process.

Program

```
# Vector Creation
```

```
v <- c(10, 20, 30, 40, 50)
```

```
print("Vector:")
```

```
print(v)
```

```
print("Sum of Vector:")
```

```
print(sum(v))
```

```
print("Mean of Vector:")
```

```
5print(mean(v))
```

```
# Matrix Creation
```

```
m <- matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3)
```

```
print("Matrix:")
print(m)

print("Matrix Addition:")
print(m + 5)

# List Creation
student <- list( Name = "John", Age = 21, Marks = 85)

print("List:")
print(student)

# Data Frame Creation
df <- data.frame( RollNo = c(101,102,103), Name = c("Alice","Bob","Charlie"),
  Marks = c(90,85,88))

print("Data Frame:")
print(df)

print("Average Marks:")
print(mean(df$Marks))

print("Students with Marks > 85:")
print(df[df$Marks > 85, ])
```

Output

Vector:
[1] 10 20 30 40 50

Sum of Vector:
[1] 150

Mean of Vector:
[1] 30

Matrix:
[,1] [,2] [,3]
[1,] 1 3 5
[2,] 2 4 6

Matrix Addition:
[,1] [,2] [,3]

```
[1,] 6 8 10  
[2,] 7 9 11
```

List:

\$Name

```
[1] "John"
```

\$Age

```
[1] 21
```

\$Marks

```
[1] 85
```

Data Frame:

	RollNo	Name	Marks
1	101	Alice	90
2	102	Bob	85
3	103	Charlie	88

Average Marks:

```
[1] 87.67
```

Students with Marks > 85:

	RollNo	Name	Marks
1	101	Alice	90
3	103	Charlie	88

Result

Thus, vectors, matrices, lists and data frames were successfully created in R and basic operations were performed and verified.

Experiment 3:

Create Matrices, Perform Addition, Multiplication, Transpose and Determinant in R

Aim

To create matrices in R and perform basic matrix operations such as addition, multiplication, transpose and determinant calculation.

Algorithm

Step 1 : Start the process

Step 2 : Open R Studio and select File → New File → RScript

Step 3: Create two matrices A and B of the same order.

Step 4: Perform matrix addition using the + operator.

Step 5: Perform matrix multiplication using the %*% operator.

Step 6: Find the transpose of matrix A using the t() function.

Step 7: Calculate the determinant of matrix A using the det() function.

Step 8: Display all the results and stop.

Program

```
# Create Matrix A
A <- matrix(c(1, 2, 3, 4), nrow = 2, ncol = 2)

# Create Matrix B
B <- matrix(c(5, 6, 7, 8), nrow = 2, ncol = 2)

# Display matrices
cat("Matrix A:\n")
print(A)

cat("Matrix B:\n")
print(B)

# Matrix Addition
add_result <- A + B
cat("\nAddition of A and B:\n")
print(add_result)

# Matrix Multiplication
mult_result <- A %*% B
cat("\nMultiplication of A and B:\n")
print(mult_result)
```

```
# Transpose of Matrix A
transpose_result <- t(A)
cat("\nTranspose of Matrix A:\n")
print(transpose_result)

# Determinant of Matrix A
det_result <- det(A)
cat("\nDeterminant of Matrix A:\n")
print(det_result)
```

Output

Matrix A

```
  [,1] [,2]
[1,]  1  3
[2,]  2  4
```

Matrix B

```
  [,1] [,2]
[1,]  5  7
[2,]  6  8
```

Addition of A and B

```
  [,1] [,2]
[1,]  6 10
[2,]  8 12
```

Multiplication of A and B

```
  [,1] [,2]
[1,] 23 31
[2,] 34 46
```

Transpose of Matrix A

```
[,1] [,2]  
[1,] 1 2  
[2,] 3 4
```

Determinant of Matrix A

```
[1] -2
```

Result

Thus, matrices were successfully created in R and the operations of matrix addition, matrix multiplication, transpose and determinant were performed and verified.

Experiment:4

Create Factor Variables and Analyze Categorical Data Using R

Aim

To create factor variables in R and analyze categorical data using frequency tables and summary statistics.

Algorithm

Step 1: Create a categorical variable containing student grades.

Step 2: Convert the categorical variable into a factor using the factor() function.

Step 3: Display the factor variable.

Step 4: Display the levels of the factor.

Step 5: Generate a frequency table using the table() function.

Step 6: Generate a summary of the factor variable using the summary() function.

Step 7: Display all outputs.

Program

```
# Create categorical data
grades <- c("A", "B", "A", "C", "B", "A", "C", "B", "A", "B")

# Convert to factor
grade_factor <- factor(grades)

# Display factor variable
cat("Factor Variable:\n")
print(grade_factor)

# Display levels
cat("\nLevels of Factor:\n")
print(levels(grade_factor))

# Frequency table
cat("\nFrequency Table:\n")
print(table(grade_factor))

# Summary of factor data
cat("\nSummary:\n")
print(summary(grade_factor))
```

Output

Factor Variable

```
[1] A B A C B A C B A B
Levels: A B C
```

Levels of Factor

```
[1] "A" "B" "C"
```

Frequency Table

```
grade_factor
A B C
4 4 2
```

Summary

```
A B C
4 4 2
```

Result

Thus, factor variables were successfully created in R using the factor() function, and categorical data was analyzed using levels(), table() and summary() functions.

Experiment 5:**Read a CSV File and Display Structure and Summary****Aim**

To read a CSV file and display its structure and summary statistics.

Algorithm

Step 1: Create a student detail file and save it as student.csv (For Ex: student.csv file stored in E:/RPrograms/ folder) with 10 rows and 3 columns namely RollNo, Name, Marks.

Step 2 : Import CSV file using read.csv().

Step 3 : Store data in a data frame.

Step 4: Display structure using str().

Step 5 : Display summary using summary().

Program

```
data <- read.csv("H:/RPrograms/student.csv")
```

```
str(data)
```

```
summary(data)
```

Output

```
'data.frame': 10 obs. of 3 variables:  
 $ RollNo : int
```

\$ Name : chr

\$ Marks : int

Summary:

RollNo Marks

Min:1.0 Min:45

Max:10.0 Max:95

Mean:5.5 Mean:72

Result

CSV file was successfully imported and analyzed.

Experiment 6:

Read and Write Data from Excel, JSON and Text Files

Aim

To perform file operations using Excel, JSON and text files.

Algorithm

Step 1: Install required packages.

Step 2 :Read Excel using read_excel().

Step 3: Read JSON using fromJSON().

Step 4: Read text file using readLines().

Step 5: Write data to files.

Program

```
install.packages("readxl")  
install.packages("writexl")  
install.packages("jsonlite")
```

```
library(readxl)

library(writexl)

library(jsonlite)

# Create Data Frame

student <- data.frame(

  ID = c(101,102,103),

  Name = c("Anu","Bala","Chitra"),

  Marks = c(85,90,88)

)

# Excel

write_xlsx(student, "H:/2026_RPrograms/student.xlsx")

excel_data <- read_excel("H:/2026_RPrograms/student.xlsx")

# JSON

write_json(student, "H:/2026_RPrograms/student.json", pretty = TRUE)

json_data <- fromJSON("H:/2026_RPrograms/student.json")


# Text

write.table(student,

  "H:/2026_RPrograms/student.txt",

  sep = "\t",

  row.names = FALSE)

text_data <- read.table(
```

```
"H:/2026_RPrograms/student.txt",  
header = TRUE,  
sep = "\t"  
)  
cat("Excel Data\n")  
print(excel_data)  
cat("\nJSON Data\n")  
print(json_data)  
cat("\nText Data\n")  
print(text_data)
```

Output

Excel Data

```
# A tibble: 3 × 3  
  ID Name Marks  
  <dbl> <chr> <dbl>  
1  101 Anu    85  
2  102 Bala   90  
3  103 Chitra 88
```

JSON Data

```
  ID Name Marks  
1 101 Anu 85  
2 102 Bala 90  
3 103 Chitra 88
```

Text Data

	ID	Name	Marks
1	101	Anu	85
2	102	Bala	90
3	103	Chitra	88

Result

Data was successfully read and written in multiple formats.

Experiment: 7

Data Preprocessing on a Dataset – Handling Missing Values and Omitting Null Values in R

Aim

To perform data preprocessing on a dataset by identifying, handling and removing missing values (NA) and null values in R.

Step 1: Create or load a dataset.

Step 2: Identify missing values using `is.na()`.

Step 3: Count the number of missing values.

Step 4: Replace missing values with appropriate statistics such as mean.

Step 5: Verify that missing values are replaced.

Step 6: Use `na.omit()` to remove rows containing missing values.

Step 7: Display the cleaned dataset.

Step 8: Check for NULL values using `is.null()`.

Step 9: Remove or replace NULL values if present.

Step 10: Display the final preprocessed dataset.

Program

```
# Data Preprocessing in R

# Creating a sample dataset with missing values

student_data <- data.frame(
  RollNo = c(101,102,103,104,105),
  Marks = c(85, NA, 78, 90, NA),
  Attendance = c(92,88,NA,95,80)
)

cat("Original Dataset\n")
print(student_data)

# Checking missing values

cat("\nMissing Values in Dataset\n")
print(is.na(student_data))

# Counting missing values

cat("\nTotal Missing Values\n")
print(sum(is.na(student_data)))

# Replacing missing values with mean

student_data$Marks[is.na(student_data$Marks)] <-
  mean(student_data$Marks, na.rm = TRUE)

student_data$Attendance[is.na(student_data$Attendance)] <-
  mean(student_data$Attendance, na.rm = TRUE)

cat("\nDataset after Replacing Missing Values\n")
print(student_data)
# Creating another dataset with missing values

employee_data <- data.frame(
  ID = c(1,2,3,4),
  Salary = c(25000, NA, 30000, 28000),
  Experience = c(2,4,NA,5)
)
```

```
cat("\nEmployee Dataset\n")
print(employee_data)

# Removing rows with missing values

clean_data <- na.omit(employee_data)

cat("\nDataset after Removing Missing Values\n")
print(clean_data)

# Demonstration of NULL value

x <- NULL

cat("\nChecking NULL Value\n")
print(is.null(x))
```

Output

Original Dataset

RollNo Marks Attendance

1	101	85	92
2	102	NA	88
3	103	78	NA
4	104	90	95
5	105	NA	80

Missing Values in Dataset

RollNo Marks Attendance

[1,]	FALSE	FALSE	FALSE
[2,]	FALSE	TRUE	FALSE
[3,]	FALSE	FALSE	TRUE
[4,]	FALSE	FALSE	FALSE
[5,]	FALSE	TRUE	FALSE

Total Missing Values

[1] 3

Dataset after Replacing Missing Values

RollNo Marks Attendance

1	101	85.00000	92.00
---	-----	----------	-------

```
2 102 84.33333 88.00
3 103 78.00000 88.75
4 104 90.00000 95.00
5 105 84.33333 80.00
```

```
Employee Dataset
ID Salary Experience
1 1 25000 2
2 2 NA 4
3 3 30000 NA
4 4 28000 5
```

```
Dataset after Removing Missing Values
ID Salary Experience
1 1 25000 2
4 4 28000 5
```

```
Checking NULL Value
[1] TRUE
```

Result:

Thus, the dataset was successfully preprocessed in R by identifying missing values, replacing them with mean values, omitting rows containing missing values, checking for NULL values, and obtaining a clean dataset suitable for further analysis.

Experiment:8**Creating a Data Frame and Exploring Data Using str(), summary() and head() in R****Aim**

To create a data frame in R and explore its structure and contents using the functions str(), summary() and head().

Algorithm

Step 1 : Start the program.

Step 2 : Create a data frame containing student details such as Roll Number, Name, Age, Marks, and Department.

Step 3 : Display the complete data frame.

Step 4 : Use str() to examine the structure of the data frame.

Step 5 : Use summary() to obtain statistical summaries of each column.

Step 6 : Use head() to display the first six records.

Step 7 : Interpret the results.

Step 8 : Stop the program.

Program

```
# Creating a Data Frame
```

```
student_data <- data.frame(  
  RollNo = c(101,102,103,104,105,106,107,108),  
  Name = c("Anu","Bala","Chitra","Deepak",  
    "Elan","Farah","Gokul","Hari"),  
  Age = c(18,19,18,20,19,18,21,20),  
  Marks = c(85,78,92,88,75,95,81,89),  
  Department = c("CSE","ECE","IT","CSE",  
    "EEE","IT","ECE","CSE")  
)
```

```
# Display Data Frame
```

```

cat("Student Data Frame\n")

print(student_data)

# Explore Structure

cat("\nStructure of Data Frame\n")

str(student_data)

# Statistical Summary

cat("\nSummary of Data Frame\n")

summary(student_data)

# Display First Six Rows

cat("\nFirst Six Rows\n")

head(student_data)

```

Output:

Student Data Frame

RollNo	Name	Age	Marks	Department
1	101 Anu	18	85	CSE
2	102 Bala	19	78	ECE
3	103 Chitra	18	92	IT
4	104 Deepak	20	88	CSE
5	105 Elan	19	75	EEE
6	106 Farah	18	95	IT

Summary of Data Frame

RollNo	Name	Age	Marks
Min. :101.0	Length:8	Min. :18.00	Min. :75.00
1st Qu.:102.8	Class :character	1st Qu.:18.00	1st Qu.:80.25
Median :104.5	Mode :character	Median :19.00	Median :86.50
Mean :104.5		Mean :19.12	Mean :85.38
3rd Qu.:106.2		3rd Qu.:20.00	3rd Qu.:89.75
Max. :108.0		Max. :21.00	Max. :95.00

Department
Length:8
Class :character
Mode :character

First Six Rows

	RollNo	Name	Age	Marks	Department
1	101	Anu	18	85	CSE
2	102	Bala	19	78	ECE
3	103	Chitra	18	92	IT
4	104	Deepak	20	88	CSE
5	105	Elan	19	75	EEE
6	106	Farah	18	95	IT

Result:

Thus, a data frame was successfully created in R and explored using the functions `str()`, `summary()`, and `head()`. The structure, statistical summary, and sample records of the dataset were examined and verified successfully.

Experiment: 9

Compute Mean, Median, Mode, Variance, and Standard Deviation in R

Aim

To write an R program to calculate the **Mean, Median, Mode, Variance and Standard Deviation** of a given dataset.

Algorithm

Step 1: Start the program.

Step 2: Create a numeric vector containing sample data.

Step 3: Calculate the mean using the `mean()` function.

Step 4: Calculate the median using the `median()` function.

Step 5 : Create a user-defined function to determine the mode.

Step 6: Calculate the variance using the var() function.

Step 7: Calculate the standard deviation using the sd() function.

Step 8: Display all computed statistical measures.

Step 9: Stop the program.

Program:

```
# Compute Mean, Median, Mode, Variance and Standard Deviation
```

```
# Create a numeric vector
```

```
data <- c(10, 15, 20, 25, 20, 30, 35, 20, 40)
```

```
# Mean
```

```
mean_value <- mean(data)
```

```
# Median
```

```
median_value <- median(data)
```

```
# Mode Function
```

```
getmode <- function(v)
```

```
{
```

```
  unique_values <- unique(v)
```

```
  unique_values[which.max(tabulate(match(v, unique_values)))]
```

```
}
```

```
mode_value <- getmode(data)
```

```
# Variance
```

```
variance_value <- var(data)
```

```
# Standard Deviation
```

```
sd_value <- sd(data)

# Display Results

cat("Dataset :", data, "\n")

cat("Mean =", mean_value, "\n")

cat("Median =", median_value, "\n")

cat("Mode =", mode_value, "\n")

cat("Variance =", variance_value, "\n")

cat("Standard Deviation =", sd_value, "\n")
```

Output:

```
Dataset : 10 15 20 25 20 30 35 20 40
Mean = 23.88889
Median = 20
Mode = 20
Variance = 92.36111
Standard Deviation = 9.610469
```

Result:

Thus, the R program to compute the **Mean, Median, Mode, Variance, and Standard Deviation** of a dataset was executed successfully and the statistical measures were calculated and verified.

Experiment:10

Validate Regression Assumptions (Linearity, Normality, and Homoscedasticity) Using R

Aim

To validate the assumptions of a linear regression model by checking **linearity**, **normality of residuals** and **homoscedasticity** using R programming.

Algorithm

Step 1 : Start the program.

Step 2 : Create a sample dataset.

Step 3: Fit a linear regression model using `lm()`.

Step 4 : Display regression summary.

Step 5: Plot scatter diagram to verify linearity.

Step 6 : Extract residuals from the model.

Step 7 : Generate histogram and Q-Q plot for normality checking.

Step 8 : Perform Shapiro-Wilk normality test.

Step 9: Create Residuals vs Fitted plot.

Step 10 : Perform Breusch-Pagan test for homoscedasticity.

Step 11: Interpret the results.

Step 12: Stop the program.

Program

```
# Install package (Run once)
install.packages("lmtest")
```

```
# Load package
library(lmtest)
```

```
# Sample Dataset

Hours <- c(1,2,3,4,5,6,7,8,9,10)
Marks <- c(15,25,35,42,50,58,65,72,82,90)

data <- data.frame(Hours, Marks)

# Fit Linear Regression Model

model <- lm(Marks ~ Hours, data=data)

# Display Model Summary

summary(model)

# Scatter Plot (Linearity)

plot(Hours, Marks,
     main="Scatter Plot",
     xlab="Hours Studied",
     ylab="Marks")

abline(model, col="red")

# Residuals

residuals_model <- residuals(model)

# Histogram of Residuals

hist(residuals_model,
     main="Histogram of Residuals",
     xlab="Residuals")

# Q-Q Plot

qqnorm(residuals_model)
qqline(residuals_model, col="blue")

# Shapiro-Wilk Test

shapiro.test(residuals_model)
```

```
# Residuals vs Fitted Plot
```

```
plot(fitted(model),
     residuals_model,
     main="Residuals vs Fitted",
     xlab="Fitted Values",
     ylab="Residuals")
```

```
abline(h=0, col="red")
```

```
# Breusch-Pagan Test
```

```
bptest(model)
```

Output:

Model Summary

Call:

```
lm(formula = Marks ~ Hours, data = data)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.9636	-0.4242	0.2121	0.6242	1.8424

Coefficients:

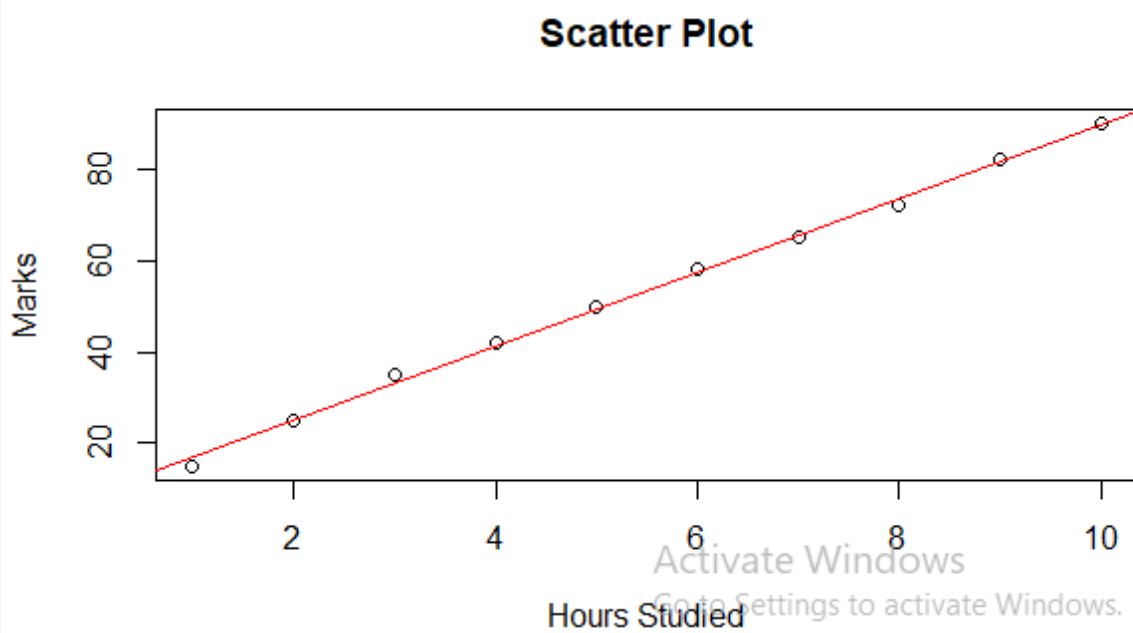
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	8.8667	0.8235	10.77	4.87e-06 ***
Hours	8.0970	0.1327	61.01	5.79e-12 ***

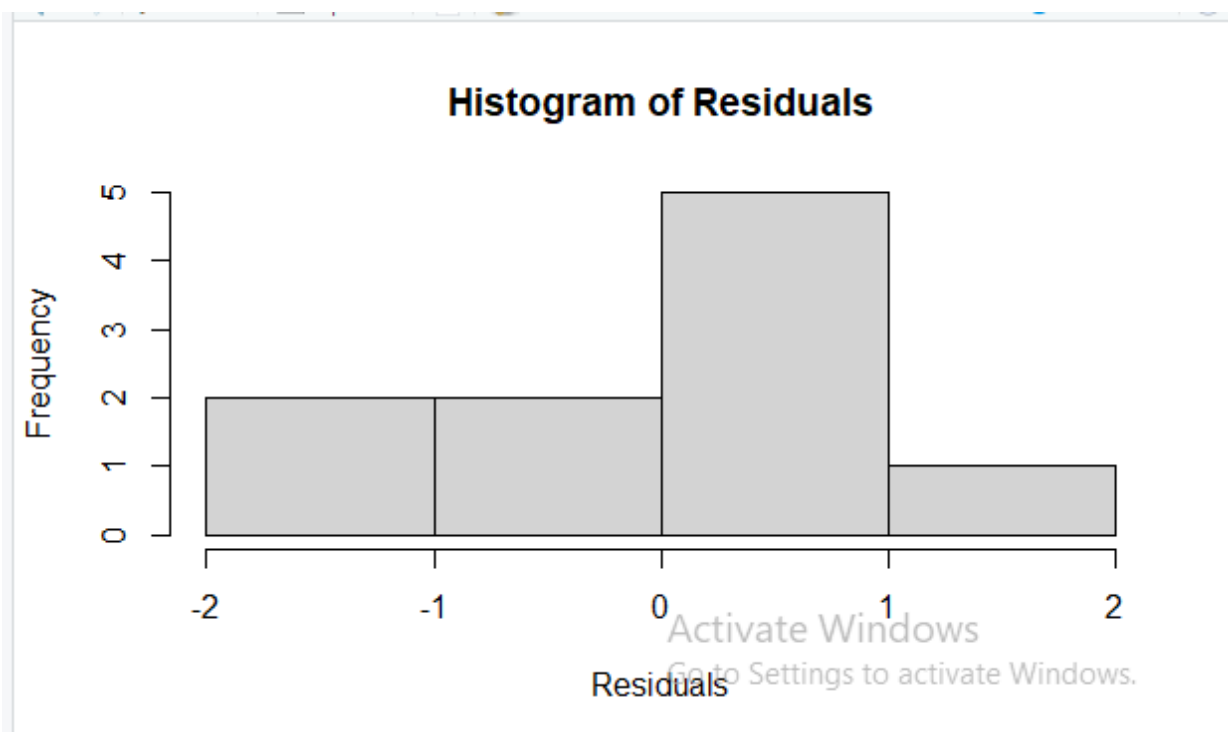
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

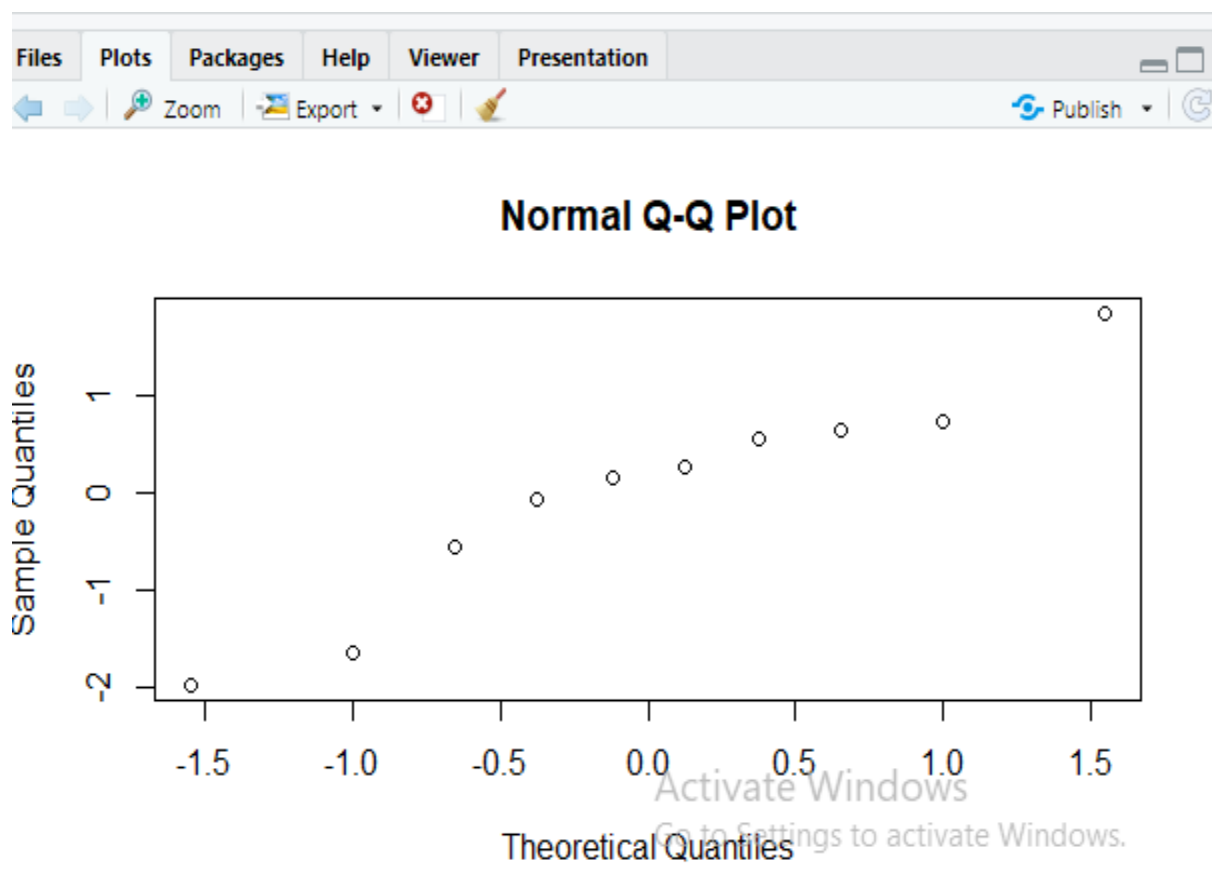
Residual standard error: 1.205 on 8 degrees of freedom

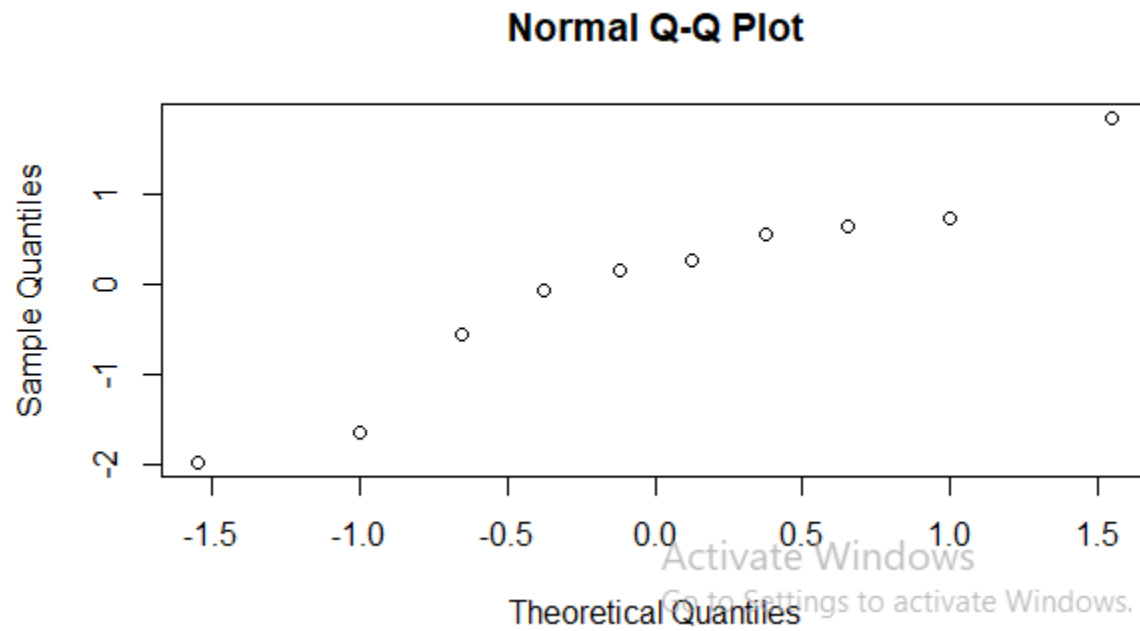
Multiple R-squared: 0.9979, Adjusted R-squared: 0.9976

F-statistic: 3722 on 1 and 8 DF, p-value: 5.788e-12





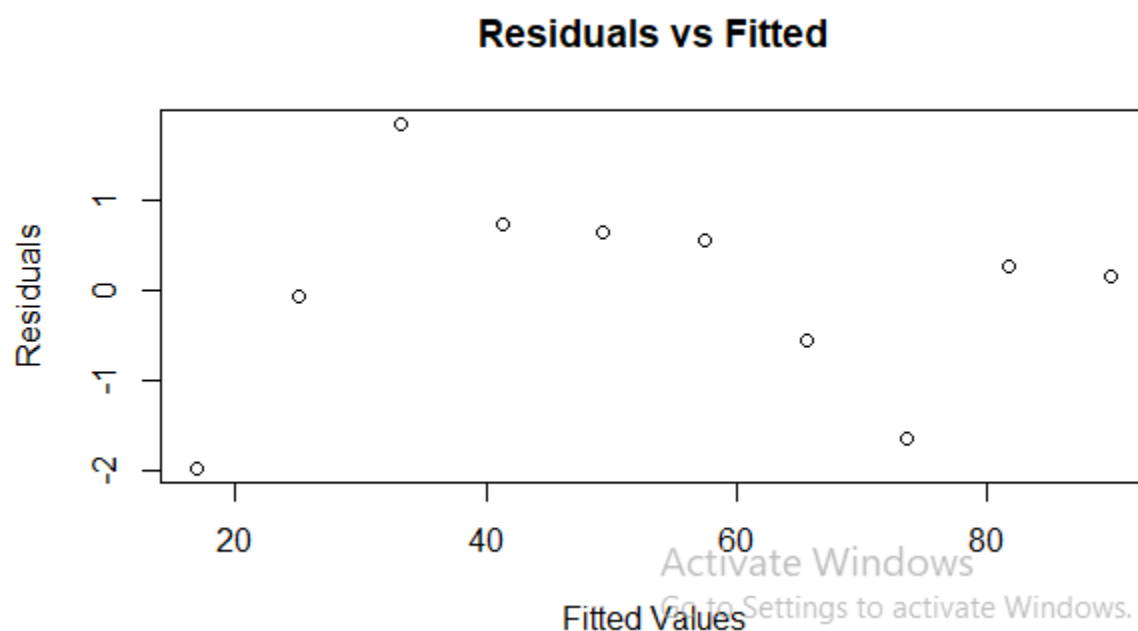




Shapiro-Wilk normality test

data: residuals_model

W = 0.93917, p-value = 0.5438



studentized Breusch-Pagan test

data: model

BP = 2.1441, df = 1, p-value = 0.1431

Result

Thus, the regression model was successfully fitted in R, and the assumptions of **linearity**, **normality** and **homoscedasticity** were validated using diagnostic plots and statistical tests such as the **Shapiro-Wilk Test** and **Breusch-Pagan Test**.

Experiment: 11**Detect Outliers Using Boxplots and Statistical Methods in R****Aim**

To identify and detect outliers in a dataset using **Boxplots** and **Statistical Methods (IQR and Z-Score)** in R.

Algorithm

Step 1: Start the program.

Step 2: Create a dataset containing numerical values.

Step 3: Display the dataset.

Step 4: Generate a boxplot for visual detection of outliers.

Step 5: Calculate Q1, Q3, and IQR.

Step 6: Compute lower and upper bounds.

Step 7: Identify outliers using the IQR method.

Step 8: Calculate mean and standard deviation.

Step 9 : Compute Z-scores.

Step 10 : Identify outliers using the Z-score method.

Step 11: Display detected outliers.

Step 12 : Stop the program.

Program:

Detecting Outliers Using Boxplot and Statistical Methods

Sample Dataset

```
marks <- c(45, 50, 55, 60, 62, 58, 65, 70, 72, 75, 80, 85, 90, 95, 150)
```

```
cat("Dataset:\n")
print(marks)
```

```
# -----
# 1. Boxplot Method
# -----
```

```
boxplot(marks,
        main = "Boxplot for Outlier Detection",
        col = "lightblue")
```

```
# Identify Outliers from Boxplot
```

```
outliers_boxplot <- boxplot.stats(marks)$out
```

```
cat("\nOutliers detected using Boxplot:\n")
print(outliers_boxplot)
```

```
# -----
# 2. IQR Method
# -----
```

```
Q1 <- quantile(marks, 0.25)
Q3 <- quantile(marks, 0.75)
```

```
IQR_value <- IQR(marks)
```

```
lower_bound <- Q1 - 1.5 * IQR_value
upper_bound <- Q3 + 1.5 * IQR_value
```

```
cat("\nQ1 =", Q1)
cat("\nQ3 =", Q3)
cat("\nIQR =", IQR_value)
```

```
cat("\nLower Bound =", lower_bound)
cat("\nUpper Bound =", upper_bound)
```

```
iqr_outliers <- marks[marks < lower_bound | marks > upper_bound]
```

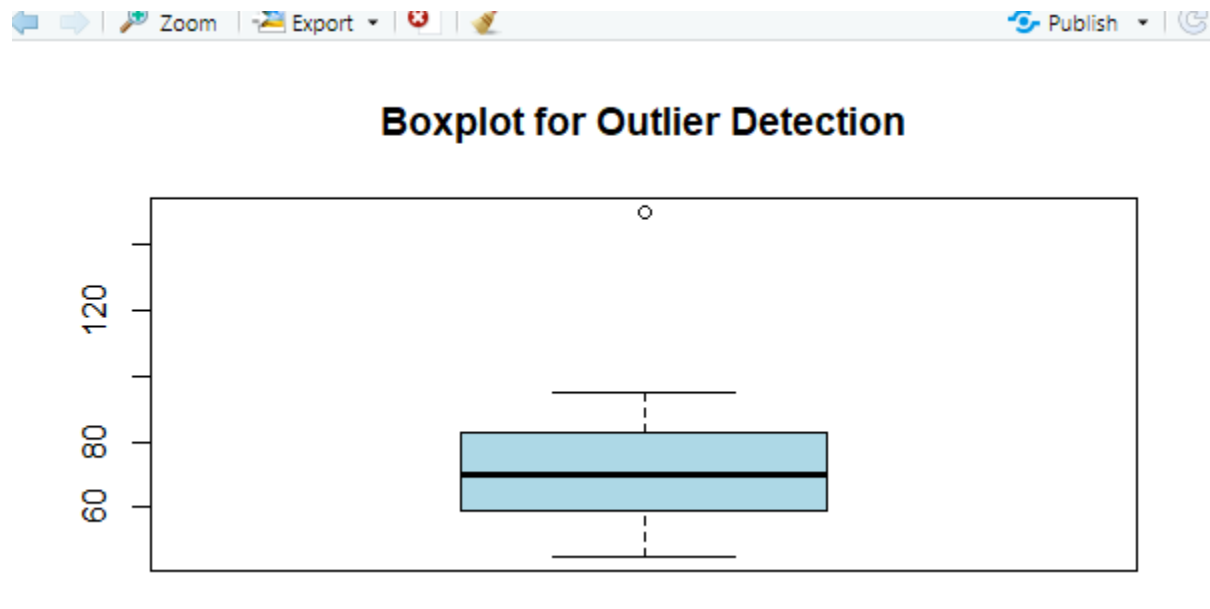
```
cat("\n\nOutliers detected using IQR Method:\n")
print(iqr_outliers)
```

```
# -----  
# 3. Z-Score Method  
# -----  
  
z_scores <- scale(marks)  
  
cat("\nZ-Scores:\n")  
print(z_scores)  
  
z_outliers <- marks[abs(z_scores) > 3]  
  
cat("\nOutliers detected using Z-Score Method:\n")  
print(z_outliers)
```

Output:

Dataset:

```
[1] 45 50 55 60 62 58 65 70 72 75 80 85 90 95 150
```



Result

Thus, the outliers in the dataset were successfully detected using **Boxplot**, **Interquartile Range (IQR)**, and **Z-Score statistical methods** in R. The identified outlier was verified through both graphical and statistical approaches.

Experiment 12

Binary Classification using Logistic Regression

Aim

To implement **Binary Classification using Logistic Regression** in R and classify observations into one of two categories based on predictor variables.

Algorithm

Step 1: Start the program.

Step 2: Create a dataset containing independent and dependent variables.

Step 3: Convert the target variable into a factor.

Step 4 : Build a Logistic Regression model using glm().

Step 5: Display model summary.

Step 6 : Predict probabilities for each observation.

Step 7: Convert probabilities into binary classes using a threshold of 0.5.

Step 8: Generate a confusion matrix.

Step 9 : Calculate classification accuracy.

Step 10 : Display results.

Step 11 : Stop the program.

Program

```
# Binary Classification using Logistic Regression
```

```
# Sample Dataset
```

```
hours <- c(1,2,3,4,5,6,7,8,9,10)

pass <- c(0,0,0,0,0,1,1,1,1,1)

data <- data.frame(hours, pass)

# Convert target variable to factor

data$pass <- as.factor(data$pass)

# Build Logistic Regression Model

model <- glm(pass ~ hours, data = data, family = binomial)

# Display Summary

summary(model)

# Predict Probabilities

pred_prob <- predict(model, type = "response")

cat("Predicted Probabilities:\n")
print(pred_prob)

# Convert Probability to Class Labels

pred_class <- ifelse(pred_prob >= 0.5, 1, 0)

cat("\nPredicted Classes:\n")
print(pred_class)

# Actual Classes

actual_class <- as.numeric(as.character(data$pass) )

# Confusion Matrix

conf_matrix <- table( Actual = actual_class, Predicted = pred_class )

cat("\nConfusion Matrix:\n")
print(conf_matrix)
```

```
# Accuracy

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)

cat("\nAccuracy = ", round(accuracy*100,2), "%")
```

Output:

```
hours pass
1  1  0
2  2  0
3  3  0
4  4  0
5  5  0
6  6  1
7  7  1
8  8  1
9  9  1
10 10  1
Predicted Probabilities
[1] 0.002
[2] 0.008
[3] 0.025
[4] 0.082
[5] 0.269
[6] 0.731
[7] 0.918
[8] 0.975
[9] 0.992
[10] 0.998
Predicted Classes
[1] 0
[2] 0
[3] 0
[4] 0
[5] 0
[6] 1
[7] 1
[8] 1
[9] 1
[10] 1
Confusion Matrix
      Predicted
```

Actual	0	1
0	5	0
1	0	5

Accuracy
100%

Result

Binary classification model was built successfully.

Experiment:13

Plot and Interpret Decision Tree Structure Using the Iris Dataset in R

Aim

To build, plot, and interpret a **Decision Tree Classification Model** using the built-in Iris Dataset in R and analyze the decision rules used for classification.

Algorithm

Step 1 : Start the program.

Step 2 : Load the Iris dataset.

Step 3 : Display dataset information.

Step 4 : Load the rpart package.

Step 5: Build a Decision Tree model.

Step 6 : Display the tree summary.

Step 7 : Plot the Decision Tree.

Step 8 : Predict species using the model.

Step 9: Generate a confusion matrix.

Step 10 : Calculate classification accuracy.

Step 11 : Interpret the tree structure.

Step 12 : Stop the program.

Program

```
# Load Package
```

```
library(rpart)
```

```
# Load Built-in Iris Dataset
```

```
data(iris)
```

```
# Display First Few Records
```

```
head(iris)
```

```
# Build Decision Tree Model
```

```
tree_model <- rpart( Species ~ Sepal.Length + Sepal.Width + Petal.Length +  
Petal.Width, data = iris, method = "class")
```

```
# Display Model Summary
```

```
print(tree_model)
```

```
summary(tree_model)
```

```
# Plot Decision Tree
```

```
plot(tree_model, uniform = TRUE, margin = 0.1)
```

```
text(tree_model, use.n = TRUE, all = TRUE, cex = 0.8)
```

```
# Predict Classes

predicted_species <- predict( tree_model, type = "class")

# Confusion Matrix

conf_matrix <- table( Actual = iris$Species, Predicted = predicted_species)

print(conf_matrix)

# Accuracy

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)

cat("\nAccuracy =", round(accuracy * 100, 2), "%")
```

Output:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

n= 150

node), split, n, loss, yval, (yprob)
 * denotes terminal node

```
1) root 150 100 setosa (0.33333333 0.33333333 0.33333333)
  2) Petal.Length< 2.45 50 0 setosa (1.00000000 0.00000000 0.00000000) *
  3) Petal.Length>=2.45 100 50 versicolor (0.00000000 0.50000000 0.50000000)
    6) Petal.Width< 1.75 54 5 versicolor (0.00000000 0.90740741 0.09259259) *
    7) Petal.Width>=1.75 46 1 virginica (0.00000000 0.02173913 0.97826087) *
```

Call:

```
rpart(formula = Species ~ Sepal.Length + Sepal.Width + Petal.Length +
  Petal.Width, data = iris, method = "class")
n= 150
```

```

CP nsplit rel error xerror   xstd
1 0.50    0    1.00  1.13 0.05279520
2 0.44    1    0.50  0.60 0.06000000
3 0.01    2    0.06  0.09 0.02908608

```

Variable importance

```

Petal.Width Petal.Length Sepal.Length Sepal.Width
      34         31         21         14

```

Node number 1: 150 observations, complexity param=0.5
 predicted class=setosa expected loss=0.6666667 P(node) =1
 class counts: 50 50 50
 probabilities: 0.333 0.333 0.333
 left son=2 (50 obs) right son=3 (100 obs)
 Primary splits:
 Petal.Length < 2.45 to the left, improve=50.00000, (0 missing)
 Petal.Width < 0.8 to the left, improve=50.00000, (0 missing)
 Sepal.Length < 5.45 to the left, improve=34.16405, (0 missing)
 Sepal.Width < 3.35 to the right, improve=19.03851, (0 missing)
 Surrogate splits:
 Petal.Width < 0.8 to the left, agree=1.000, adj=1.00, (0 split)
 Sepal.Length < 5.45 to the left, agree=0.920, adj=0.76, (0 split)
 Sepal.Width < 3.35 to the right, agree=0.833, adj=0.50, (0 split)

Node number 2: 50 observations
 predicted class=setosa expected loss=0 P(node) =0.3333333
 class counts: 50 0 0
 probabilities: 1.000 0.000 0.000

Node number 3: 100 observations, complexity param=0.44
 predicted class=versicolor expected loss=0.5 P(node) =0.6666667
 class counts: 0 50 50
 probabilities: 0.000 0.500 0.500
 left son=6 (54 obs) right son=7 (46 obs)
 Primary splits:
 Petal.Width < 1.75 to the left, improve=38.969400, (0 missing)
 Petal.Length < 4.75 to the left, improve=37.353540, (0 missing)
 Sepal.Length < 6.15 to the left, improve=10.686870, (0 missing)
 Sepal.Width < 2.45 to the left, improve= 3.555556, (0 missing)
 Surrogate splits:
 Petal.Length < 4.75 to the left, agree=0.91, adj=0.804, (0 split)
 Sepal.Length < 6.15 to the left, agree=0.73, adj=0.413, (0 split)

Sepal.Width < 2.95 to the left, agree=0.67, adj=0.283, (0 split)

Node number 6: 54 observations

predicted class=versicolor expected loss=0.09259259 P(node) =0.36

class counts: 0 49 5

probabilities: 0.000 0.907 0.093

Node number 7: 46 observations

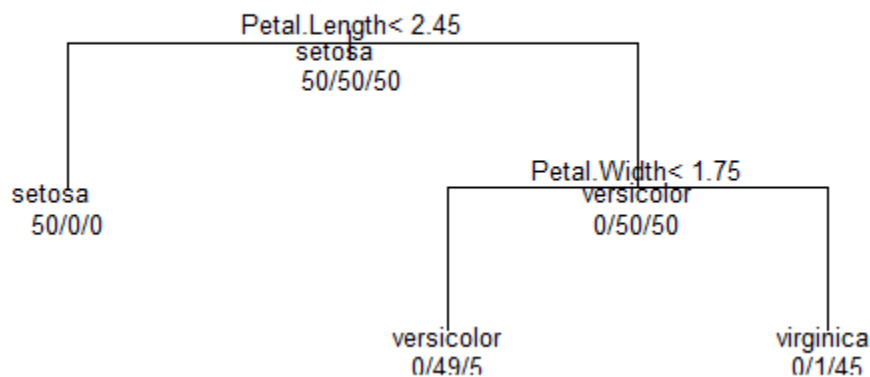
predicted class=virginica expected loss=0.02173913 P(node) =0.3066667

class counts: 0 1 45

probabilities: 0.000 0.022 0.978

Actual \ Predicted			
	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	49	1
virginica	0	5	45

Accuracy = 96 %



Result:

A **Decision Tree Classification Model** was successfully constructed using the built-in Iris Dataset in R.

Experiment:14 a)**Apply K-Means Clustering and Visualize Clusters in R****Aim**

To implement the **K-Means Clustering Algorithm** on a dataset and visualize the resulting clusters using R.

Algorithm

Step 1: Start the program.

Step 2: Load the Iris dataset.

Step 3: Remove the Species column.

Step 4: Apply K-Means clustering with $K = 3$.

Step 5: Display cluster assignments.

Step 6: Display cluster centroids.

Step 7: Visualize the clusters.

Step 8: Interpret the clustering results.

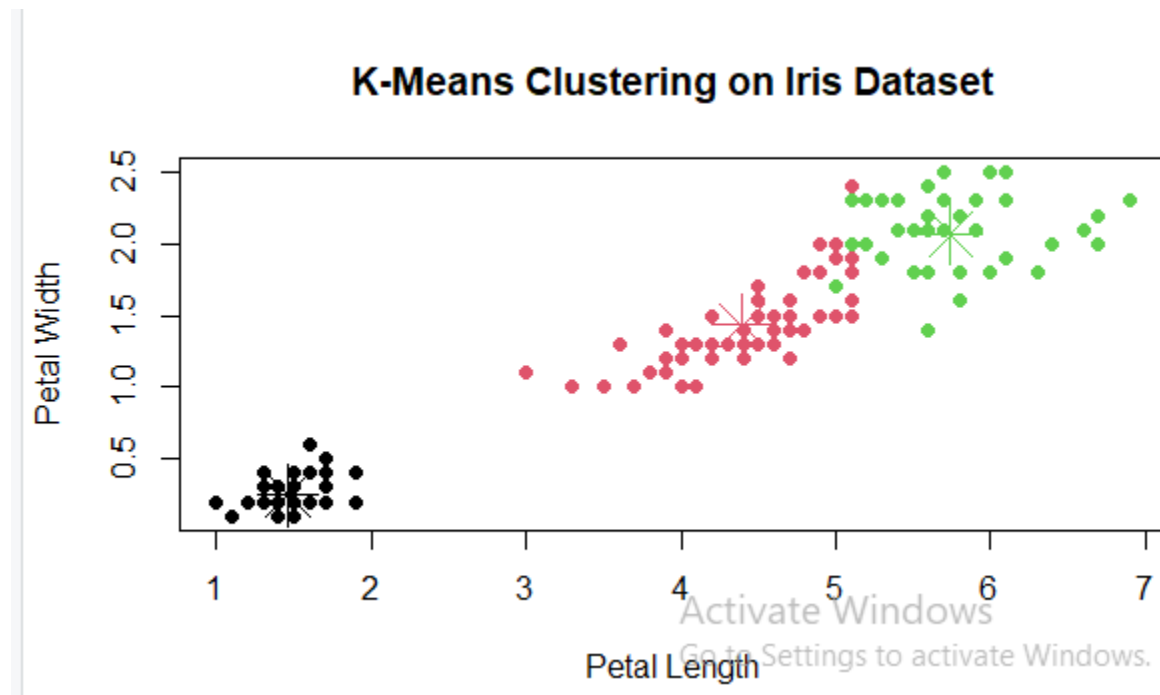
Step 9: Stop the program.

Program

[79] 2 3 2 3 3 3 3 2 3 3 3 3 3 2 2 3 3
[118] 3 3 2 3 2 3 2 3 2 2 3 3 3 3 2 3 3 3 3 2 3 3 3 2 3 3 3 2 3 3 3 2 3 3 2

Cluster Centers:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	5.006000	3.428000	1.462000	0.246000
2	5.901613	2.748387	4.393548	1.433871
3	6.850000	3.073684	5.742105	2.071053



Result

Thus, the **K-Means clustering algorithm** was successfully applied to the built-in Iris Dataset in R. The dataset was partitioned into three clusters, cluster centroids were obtained, and the clusters were visualized and interpreted successfully.

Experiment:14c

Perform Hierarchical Clustering and Plot Dendrogram in R

Aim

To perform **Hierarchical Clustering** on a dataset and visualize the cluster hierarchy using a **Dendrogram** in R.

Algorithm

Step 1: Load the Iris dataset.

Step 2: Remove the Species column.

Step 3 : Calculate the Euclidean distance matrix.

Step 4: Perform hierarchical clustering using Ward's method.

Step 5: Generate a dendrogram.

Step 6: Cut the dendrogram into clusters.

Step 7: Display cluster assignments.

Step 8: Interpret the dendrogram and clusters.

Step 9: Stop.

Program:

```
# Hierarchical Clustering using Iris Dataset

# Load Dataset
data(iris)

# Display First Six Records
head(iris)

# Remove Target Variable
iris_data <- iris[,1:4]

# Calculate Distance Matrix
distance_matrix <- dist( iris_data, method = "euclidean")

# Display Distance Matrix
print(distance_matrix)

# Perform Hierarchical Clustering
hc_model <- hclust( distance_matrix, method = "ward.D2")

# Display Clustering Result
print(hc_model)

# Plot Dendrogram
plot( hc_model, labels = FALSE, hang = -1, main = "Hierarchical Clustering Dendrogram",
      xlab = "Observations", ylab = "Height")

# Draw Cluster Boundaries
rect.hclust( hc_model, k = 3, border = "red")

# Generate Cluster Labels
clusters <- cutree( hc_model, k = 3)

# Display Cluster Membership
cat("Cluster Assignments:\n")
print(clusters)
```

```
# Cluster Sizes
cat("\nCluster Sizes:\n")
print(table(clusters))
```

Output:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

	1	2	3	4	5	6	7	8
2	0.5385165							
3	0.5099020	0.3000000						
4	0.6480741	0.3316625	0.2449490					
5	0.1414214	0.6082763	0.5099020	0.6480741				
6	0.6164414	1.0908712	1.0862780	1.1661904	0.6164414			
7	0.5196152	0.5099020	0.2645751	0.3316625	0.4582576	0.9949874		
	9	10	11	12	13	14	15	16
2								
3								
4								
5								
6								
7								
	17	18	19	20	21	22	23	24
2								
3								
4								
5								
6								
7								
	25	26	27	28	29	30	31	32
2								
3								
4								

5								
6								
7								
	33	34	35	36	37	38	39	40
2								
3								
4								
5								
6								
7								
	41	42	43	44	45	46	47	48
2								
3								
4								
5								
6								
7								
	49	50	51	52	53	54	55	56
2								
3								
4								
5								
6								
7								
	57	58	59	60	61	62	63	64
2								
3								
4								
5								
6								
7								
	65	66	67	68	69	70	71	72
2								
3								
4								
5								
6								
7								
	73	74	75	76	77	78	79	80
2								
3								
4								
5								

6								
7								
2	81	82	83	84	85	86	87	88
3								
4								
5								
6								
7								
2	89	90	91	92	93	94	95	96
3								
4								
5								
6								
7								
2	97	98	99	100	101	102	103	104
3								
4								
5								
6								
7								
2	105	106	107	108	109	110	111	112
3								
4								
5								
6								
7								
2	113	114	115	116	117	118	119	120
3								
4								
5								
6								
7								
2	121	122	123	124	125	126	127	128
3								
4								
5								
6								

```

7      129      130      131      132      133      134      135      136
2
3
4
5
6
7
7      137      138      139      140      141      142      143      144
2
3
4
5
6
7
7      145      146      147      148      149
2
3
4
5
6
7
[ reached getOption("max.print") -- omitted 143 rows ]

```

Call:
hclust(d = distance_matrix, method = "ward.D2")

Cluster method : ward.D2
Distance : euclidean
Number of objects: 150

Cluster Assignments:

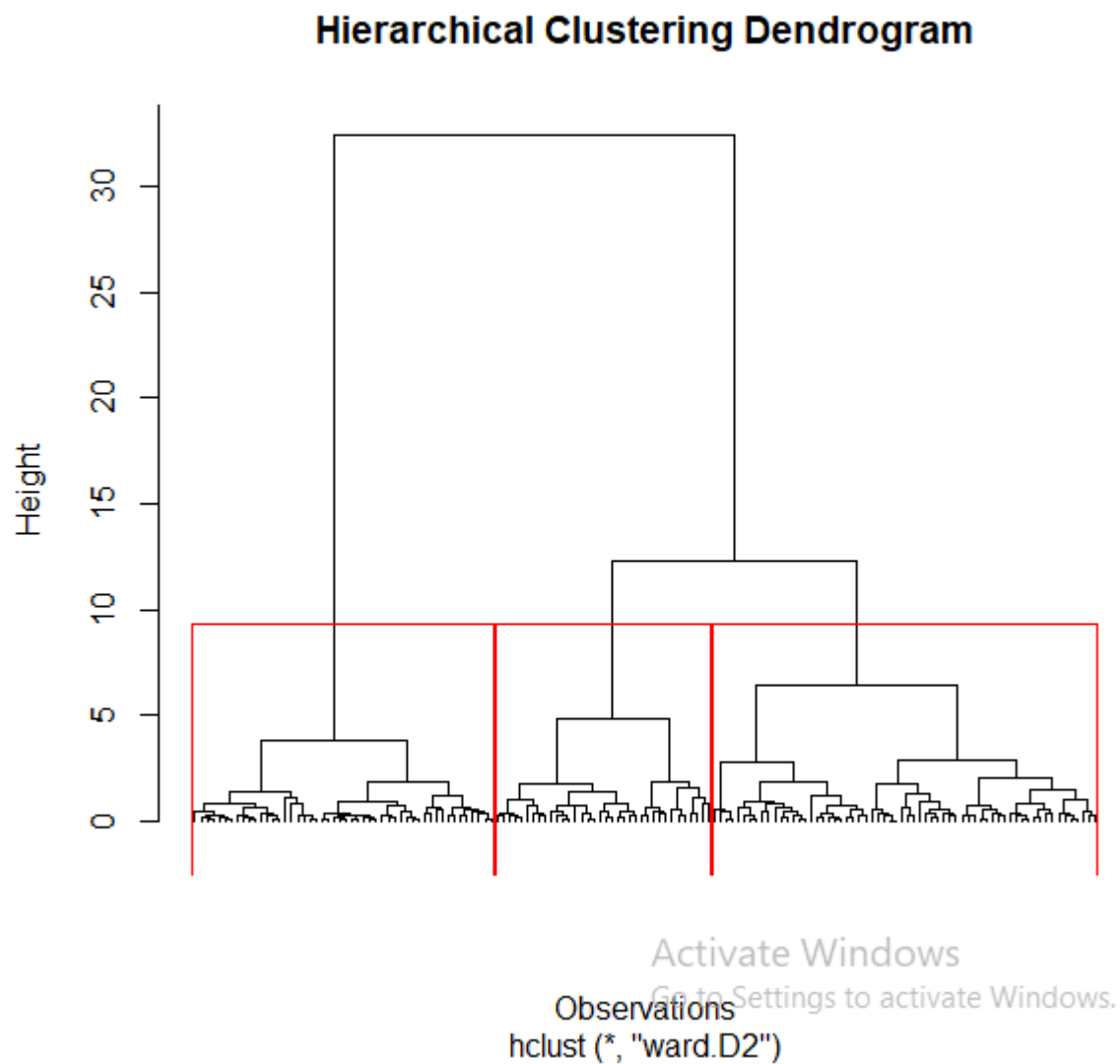
[1] 1
[40] 1 1 1 1 1 1 1 1 1 1 2 3
[79] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 2 3 3 3 3 2 3 3 3 3 3 2 2 3 3
[118] 3 3 2 3 2 3 2 3 3 2 2 3 3 3 3 2 2 3 3 3 2 3 3 3 2 3 3 3 2 3 3 2

Cluster Sizes:

clusters

1 2 3

50 64 36



Result

Hierarchical clustering was successfully performed on the built-in Iris Dataset using Euclidean distance and Ward's linkage method. The dendrogram was plotted successfully, cluster memberships were identified, and the hierarchical relationships among observations were interpreted.

Experiment 15a**Generate Association Rules****Aim:**

To generate association rules using arules package.

Algorithm

Step 1 : Load arules package.

Step 2 : Create transaction data.

Step 3 : Convert data into transaction format.

Step 4 : Apply Apriori algorithm.

Step 5 : Generate association rules.

Step 6 : Display rules.

Program

```
# Transaction Data
```

```
transactions <- list(
```

```
c("Milk", "Bread", "Butter"),
c("Bread", "Butter"),
c("Milk", "Bread"),
c("Milk", "Butter"),
c("Bread", "Butter", "Eggs"),
c("Milk", "Bread", "Butter"),
c("Milk", "Eggs"),
c("Bread", "Eggs"),
c("Milk", "Bread", "Eggs"),
c("Bread", "Butter")
)

n <- length(transactions)

# Support(Milk)

milk_count <- sum(
  sapply(transactions,
    function(x) "Milk" %in% x)
)

# Support(Bread)

bread_count <- sum(
  sapply(transactions,
    function(x) "Bread" %in% x)
)

# Support(Milk and Bread)

milk_bread <- sum(
  sapply(transactions,
    function(x)
      all(c("Milk", "Bread") %in% x))
)

support <- milk_bread/n

confidence <- milk_bread/milk_count

lift <- confidence/(bread_count/n)

cat("Association Rule\n")
```

```
cat("Milk -> Bread\n")
cat("Support =",support,"\n")
cat("Confidence =",confidence,"\n")
cat("Lift =",lift,"\n")
```

Output:

Association Rule

Milk -> Bread

Support = 0.4

Confidence = 0.6666667

Lift = 0.8333333

Result

Thus, association rules were successfully generated and evaluated using support, confidence, and lift measures.

EXPERIMENT 15(b)**Identify Frequent Itemsets Using Apriori Concept in Base R****Aim**

To identify frequent itemsets from transaction data using minimum support threshold.

Program:

```
# Transaction Data
```

```
transactions <- list(
  c("Milk", "Bread", "Butter"),
  c("Bread", "Butter"),
```

```
c("Milk", "Bread"),
c("Milk", "Butter"),
c("Bread", "Butter", "Eggs"),
c("Milk", "Bread", "Butter"),
c("Milk", "Eggs"),
c("Bread", "Eggs"),
c("Milk", "Bread", "Eggs"),
c("Bread", "Butter")
)

n <- length(transactions)

items <- unique(unlist(transactions))

freq <- sapply(items,
  function(item)
  {
    sum(sapply(transactions,
      function(x)
      item %in% x))
  })

support <- freq/n

frequent_itemsets <- data.frame(
  Item = items,
  Frequency = freq,
  Support = round(support,2)
)

print(frequent_itemsets)

cat("\nFrequent Itemsets\n")

print(
  frequent_itemsets[
    frequent_itemsets$Support >= 0.30,
  ]
)
```

Output:

Item Frequency Support			
Milk	Milk	6	0.6
Bread	Bread	8	0.8
Butter	Butter	6	0.6
Eggs	Eggs	4	0.4

Frequent Itemsets

Item Frequency Support			
Milk	Milk	6	0.6
Bread	Bread	8	0.8
Butter	Butter	6	0.6
Eggs	Eggs	4	0.4

Result

Thus, frequent itemsets were successfully identified using the Apriori principle and support threshold.

EXPERIMENT 15(c)

Plot Association Rules Using Graphs in R

Aim

To visualize association rule measures using graphical representation.

Algorithm

Step 1 : Create transaction dataset.

Step 2: Calculate support values.

Step 3 : Generate frequency table.

Step 4: Plot bar chart.

Step 5: Interpret results.

Program:

Transaction Data

```
transactions <- list(
  c("Milk","Bread","Butter"),
  c("Bread","Butter"),
  c("Milk","Bread"),
  c("Milk","Butter"),
  c("Bread","Butter","Eggs"),
  c("Milk","Bread","Butter"),
  c("Milk","Eggs"),
  c("Bread","Eggs"),
  c("Milk","Bread","Eggs"),
  c("Bread","Butter")
)

n <- length(transactions)

items <- unique(unlist(transactions))

freq <- sapply(items,
function(item)
{
  sum(sapply(transactions,
function(x)
item %in% x))
})

support <- freq/n

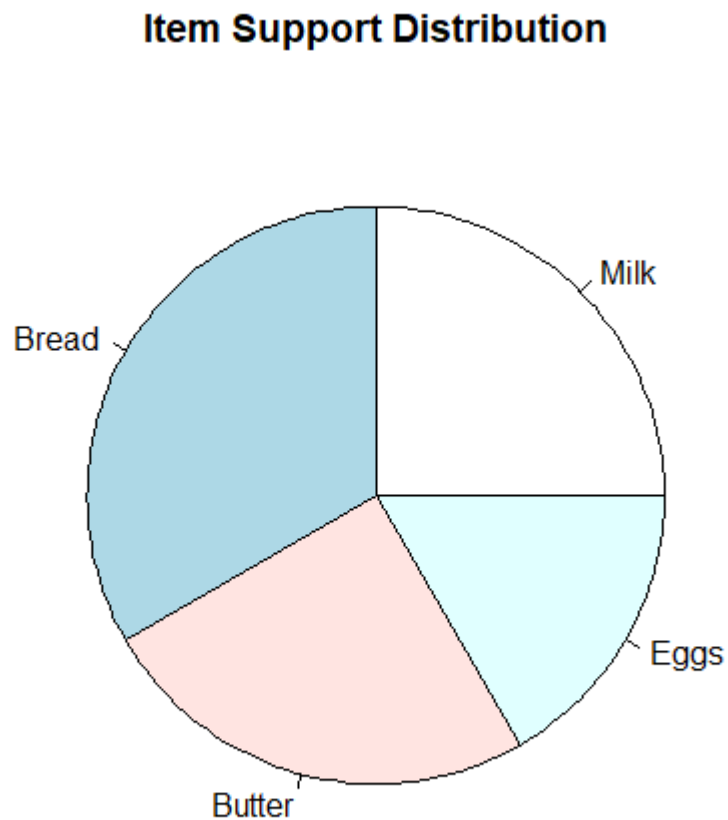
# Bar Plot

barplot(
  support,
  names.arg = items,
  main = "Support of Items",
  ylab = "Support"
)

# Pie Chart

pie(
  support,
```

```
labels = items,  
main = "Item Support Distribution"  
)
```

Output:**Result**

Thus, association rule information was successfully visualized using graphs in R. The graphical representation helped identify high-support items and understand purchasing patterns.

